

Collaborative Doc For Test 4 Studying

(Montazeri 01 and Davis 02 - Spring 2024)

We have created a document for students to collaborate on ideas for approaching problems for test 4. The aim is **to share ideas** while reducing redundant questions on the piazza. **I will, and maybe other teaching staff, though it is not required as I am volunteering my own time (i.e., do not bother the others), respond when I can. Ideally, students will help each other :)**

Paste the question under the correct category and question number, along with some initial ideas for us to work off. I will likely not respond to questions with no initial idea, as that's pretty much spoon-feeding.

Required ground rules:

1. Do not delete other people's comments.
2. Do not use any highlighting.
3. Do not share code or answers. You can say procedurally what you would like to do. However, do not give explicit code.
 - a. Example of Good Response:
 - i. Keep an integer array (let's call it `set_arr`), that has size as $\max(A)$. Set everything in `set_arr` to 0. Then for every number, $A[i]$, add one to `set_arr[A[i]]`. Then go through `set_arr[A[i]]`, and find the 1.
 - b. bad (sorry whoever I took this from):
 - i. So let's call the end number $A[\text{end}]$ and the beginning number $A[\text{beg}]$.
 - ii. If $A[\text{end}] + A[\text{beg}]$ is x , you return.
 - iii. If $A[\text{end}] + A[\text{beg}]$ is greater than x , then $\text{end} = (\text{end} + \text{beg})/2$.
 - iv. If $A[\text{end}] + A[\text{beg}]$ is less than then $\text{beg} = (\text{end} + \text{beg})/2$.
4. Be appropriate. This means not posting anything inappropriate, such as posting non-relevant materials or purposefully writing incorrect ideas.

Anyone who breaks the ground rules will have their changes reverted and blocked from this and all future collaborative documents.

Legend

Yellow Highlighted Text	A Course Staff Response.
Red Highlighted Solution	A course staff believes this response is not correct.
Orange Highlighted Solution	A course staff believes this response is correct but not the optimal solution.
Green Highlighted Solution	A course staff believes this response is correct and is an optimal solution.

Array Questions

No	Question	Ideas
1	Given a number x, find if two numbers add up to x (or the closest sum to x)	Assume: -we have an array whose size is unknown, A. -and an integer x our "target" number, x. -code is written in C. 1.sort the array A. 2.create an integer called low set to zero 3.create an integer called high set to the length of the array-1 4.whilst the low is less than the high 5. Create an int variable which sums A[low] and A[high] 6. Another Idea: For every number in array as i, loop through every number after i as j, (so i + 1). Check if the sum of any i and j is x. Should be $O(n^2)$ time complexity like selection sort. Building off the first idea: 1. Sort the array 2. Create an int low set to zero and int high set to the last element in the array. These will keep track of array indices. 3. We want to keep track of how close we are to the target value so we will create a difference variable that keeps track of this. We can initialize it to the absolute value of the difference between the sum of the values of the starting indices and the target. The goal is to find two indices where their numbers are closest to the target. If their sum is the target the difference between them should be zero (lesser difference is better). We can refer to this variable as the lowest difference.

		4. We also need to keep track of the values that create the lowest difference so we will store them in a list/pair. We can refer to this variable as the lowest pair 5. While low is less than high, if the current values observed have a lesser difference than the lowest difference, update the lowest difference and lowest pair 6. For base cases, if the current difference between the sum of the two index values and the target are less than the target, increment low to observe the next index. If the difference is greater, decrement the high index (we can do this because the array is sorted). Otherwise the lowest pair is the answer (since it passed both less than or greater than queries, it means the target and sum of the index values are equal) 7. We return the lowest pair outside the while loop as well since we are also checking for the closest sum. This solution should be $O(n \log n)$ time complexity
2	Find the median of the union of 2 sorted arrays	Assuming that you have the 2 sorted arrays already set in two different variables and you have a function that already sorts the array, let's call them array1 and array2. Then create a 3rd array (array3) and unite the two arrays into array3 and run the array3 into the sorting function. Now to get the median, if the size of array3 is odd then return the middle element from adding the first position and the size of the array divided by 2. For an even array, for example if the size of the array, without counting the first element as 0, divide it by 2 so you'd get 4; with the 4th element in the array and the next array element, $[4 + 1]$, add the two elements together and divide that with 2. Another Idea: Use two pointers to merge the sorted array. One pointer for each array, while both pointers aren't at the end, check which value is smaller at that pointer in both arrays and add the smaller value to the new merged array. Increment the pointer of whichever array had the smaller value as long as it isn't at the end of that array. Whenever either one array pointer reaches the end, just insert the remaining elements of the other array into the merged array. Now get the median value by rounding down half the size of the merged array. Time complexity should be $O(n + m)$ where n and m is size of the two sorted array respectively. Another idea (used Chatgpt to reword because I didn't explain it great): Use Binary search on a combined list

		Create a function to find the median of two sorted arrays using binary search. The function takes two arrays as input, arr1 and arr2. First, ensure that arr1 is the shorter array by swapping the arrays if necessary; this allows for more efficient binary search. Calculate the total length of the combined arrays and the half-length, storing these values in total_length and half_length, respectively. Initialize low to 0 and high to the length of arr1. Enter a while loop that continues as long as low is less than or equal to high. Within the loop, calculate the partition index i for arr1 using $(low + high) // 2$ and the partition index j for arr2 using $half_length - i$. Adjust the partitions based on the comparisons of the edge values: if $arr1[i]$ is smaller than $arr2[j-1]$, increase low to $i + 1$; if $arr1[i-1]$ is greater than $arr2[j]$, decrease high to $i - 1$. When the correct partition is found, determine the median. If the total length is odd, the median is the maximum of the left partitions; if even, it is the average of the maximum of the left partitions and the minimum of the right partitions. The time complexity should be $O(\log(n))$ where n is the minimum length between arr1 and arr2. Another Idea: Run Merge Sort on the two arrays into array 3. Initialize an int counter and go through array 3 incrementing counter by 1 at each index. If the counter is even by the end then divide by 2 to get the median index, traverse the array until you reach that index and return that value. If odd then repeat prev step but once you get the value from index put it into an int var. Add 1 to the index to get that value and add that to the var. Divide the var by 2 and return the result. $O(n \lg n) + O(n) + O(n/2) = O(n \lg n)$ Get the median of two sorted arrays, compare them, and for the array that is smaller in median value, create a new array consisting of median till the end (inclusive). For the larger median value, create a new array consisting of start till the median (inclusive). Repeat this comparison and shortening of the arrays until only 2 items are in each array, and then merge sort and find the median. Running time is $O(\log n)$ since the merge sort at the end is $O(1)$ given 4 items.
3	Find the kth smallest element in the union of two sorted arrays	Let's call the two arrays A and B. First, check that k is between 1 and the combined length of A and B inclusive. Keep three values: val and the indices i and j. Set i and j to 0. We create a loop that iterates k times, and on every iteration, we compare $A[i]$ and $B[j]$. If $A[i]$ is smaller, then we set val to $A[i]$ and increment i. Otherwise, we set val to $B[j]$ and increment j. Before incrementing indices i or j, we must check that i/j is within the bounds of A/B to avoid indexing outside of the arrays. After the loop completes, we return val. $O(k)$ Just realized this may have a problem lol

4	Above 2 questions for L sorted arrays	Question 3: The <i>val</i>, <i>i</i>, <i>j</i> solution can be generalized for any L number of arrays by using an array to store indices instead of declaring variables. Say that we have an array C containing an L amount of sorted arrays. We need to keep a variable called <i>val</i> and an integer array of zeros called <i>idxs</i>. The array represents our indices for arrays. We once again create a loop that iterates k times. On each iteration, we will find the next smallest element and the index of the array it belongs to, called <i>smallest</i> and <i>smallestIdx</i> respectively. We ignore arrays whose numbers we have already used. To do this, we can initialize <i>smallest</i> and <i>smallestIdx</i> to be <i>C[0][idxs[0]]</i> and 0 respectively. We then loop over all the arrays. Let's call our iteration variable <i>i</i>. On each iteration, we check if <i>idxs[i]</i> is less than <i>length(C[i])</i>. If not, we continue to the next iteration. If <i>C[i][idxs[i]]</i> is less than or equal to <i>smallest</i>, we set <i>smallest</i> to that value and <i>smallestIdx</i> to <i>i</i>. Note: the <i>smallest</i> variable may be redundant Then, we set <i>val</i> to <i>smallest</i> and increment <i>idxs[smallestIdx]</i>. Finally, after the outer loop completes, return <i>val</i>. Space: $O(L)$ Time: $O(k \cdot L)$ - Outer loop iterates k times, inner loop L times.
5	We discussed in class how to merge two sorted arrays. What about merging k sorted arrays?	Idea: Iterate through the k sorted arrays, merging them pairwise. Use an interval variable to keep track of this pairwise merging. Example: argument passed [A1, A2, A3, A4] First loop merge(A1, A2), merge(A3, A4) interval = 1 Second loop merge(A2, A4) interval = 2 This would have a runtime of $O(n)$ for n elements in each sorted array from calling merge and a runtime of $O(\log k)$ for k number of arrays to be merged. another idea: like the merge part of merge sort with multiple lists. have a new empty array have a pointer for each array starting at its 0th element. iterating from 1st to kth array pointer get the minimum (if there is a tie the latest array wins), add that element to the new array and increment that pointer ahead by one and repeat. i think the time complexity is $O(k \cdot n)$ if n is the sum of the number of elements in every input array, because the worst case is that all the right answer is all the arrays lined up next to each other.
6	You have an unsorted array of integers where every integer appears exactly twice, except one integer that appears once.	Idea: Create an hash table for frequency for each item in array. Looping through array, add item to hash table if it doesn't exist, else increment item value to 2

	Find it.	in hash table. At end, loop through hash table and return any element with only 1 frequency. $O(n)$ time complexity. Create an int to store the unique integer. Then, iterate through each index of the array and set the int to the xor equal of the index of the array. Addition to above ^ Once every element has been xored, the final value of the int should be the integer that appears only once as duplicate elements would cancel themselves out Adding on to the second one, make sure you don't xor itself. If the xor is ever 0, move on to the next index. Once you find the index where the xor is never 0, that is the answer. Worst runtime is $O(n^2)$
7	Given an array, describe an algo to identify the contiguous subarray with the maximum sum. For example, if the input is [1, -3, 5, -2, 9, -6, 4], the output should be [5, -2, 9]	Divide and conquer: - Divide: Split the array in the middle, into 2 subarrays - Conquer: The maximum subarray either exists in the left subarray, right subarray, or the in between two subarrays - The left and right max subarrays are found recursively - The maximum midpoint subarray between the two subarray is then found, starting at the midpoint - Create loop that iterates from midpoint down to start - Track the maximum sum you see here - Create a loop a loop that iterates from midpoint to end - Track the maximum sum you see here - The maximum midpoint subarray is then the sum of the two loops - $O(n)$ time - Thus, return the max of the 3 subarray - Performance: Behavior is similar to mergesort, thus $O(n * \log n)$ time Student response: I don't see the contradiction here? Perhaps I worded it poorly, but the third step in which the maximum array starting in the middle is found account for this. Idea: Find the two largest number Have two variable for two largest number in array. Also have two variable for the starting and ending index of sub array. Loop through each element in array, if no value in either variable or element is greater than either value, set that variable to element. Whenever a variable is updated, set the index of that value to be either the start of sub array or the end if its greater than start index. $O(n)$ time complexity. Idea:

		Create a variable to store max sum array. Have a for loop that will go through each sublength in the total length. Compare each subarray using that sublength with the max sum array variable. If it's sum is bigger, store the new subarray. Once you get to the end, return the saved array. This will execute in $O(n \log n)$ because you have to go through n times and then $\log n$ times in the 2nd loop. Go through list, continuously adding to sum. Initialize a max value variable, checking for max value during traversal. Create 2 pointers and sum up two adjacent values to put in a new array, and then increment the pointer and repeat. [1, -3, 5, -2, 9, -6, 4] -> [-2, 2, 3, 7, 3, -2], get biggest positive streak [2, 3, 7, 3] from index [2, 3, 4] => [5, -2, 9]
8	8. Given an array A of numbers, for every index i, find the nearest index j such that $A[j] > A[i]$. If none exist, report -1. Output these indices sorted by the corresponding i.	Brute force idea: Create an integer array (let's call it arr) of size(A), set every value equal to -1. Initialize a variable for the nearest index and the minimum distance. Loop through every element i, then (nested) loop through every element j (where $j = i+1$). Check if the value of j > value of i and that (j-i) is less than the minimum distance. If so, set the nearest index to j and the minimum distance to j-i. Then once finished with the first for j loop, make a second for j loop (still inside the for i loop, where $j = i-1$) that iterates backwards through the array. This for j loop checks if the value of j > value of i and that (i-j) is less than the minimum distance. If the condition is met, it then updates the nearest index to j and the minimum distance to i-j. Once the for j loops have finished, set arr[i] to the nearest index. Once out of the for i loop, return arr. Has time complexity $O(n^2)$.
9	9. Given an array of integers, move all the zeros to the right end. The order of the others doesn't matter. Use $O(1)$ extra storage..	Since order doesn't matter, you can simply keep a counter of zeros, akin to insertion sort's sorted counter, and just swap all zeros you find idea: have a pointer to the end of the array which moves forward one if at a 0. Loop through each index of the array. If there is a 0, swap it with the last non-zero integer in the array. If there are no nonzero integers after the current index, stop the loop. Executes in $O(n)$ and space complexity of $O(1)$ because you are only storing the integer value of the index. Write an index swap function As you iterate through the array, keep track of the last non-zero index (initialized as the last index in the array). When you find a zero, swap it with the last non-zero index and decrement the last non-zero index by 1. Once the

		iteration variable in the loop is greater than or equal to the last non-zero index, exit the loop. Should complete in $O(n)$ time as it only involves one array iteration loop. // idea Count the zeros in the array, when we find a zero, increment the counter for every 0 encountered, then delete the 0, increment to next index in array till next 0 and repeat. Keep count of length, when we have traversed entire length of array, push back and append the count amount of 0's at end of array. Uses $O(1)$ space since push back operation is $O(1)$, worst case the delete function takes $O(n^2)$ if every element is 0 in array.
10	10. Given an array of integers, reorder so that all the negative integers appear first, then the zeroes, and finally the positive integers. If two integers have the same sign, their order should not be changed. Use $O(1)$ extra storage.	 Have 3 variables that keep track of the current index i (equal 0), the index to check 0's location j (equal 0) and the index at the back of the array k (equal size - 1) Do a loop to go through each element on the array (so size of array) If the current index is less than 0, we swap current index i with the index that check 0's location k, increment both i and k (notice that it will swap itself until it finds a zero) If the current index is equal to 0, nothing needs to be done, continue and increase current index i only If the current index is more than 0, we swap the current index with the variable k that holds the index of the back of the array (since back of the array is positive). Decrement k so you don't overlap and keep order. This should use $O(1)$ storage because we're only using named variables This should run in $O(n)$ because of a single loop that goes through all element of an array once
11	You have a stream of numbers. Implement an algorithm to return the median of the numbers seen so far.	Idea: Using insertion sort insert each new number into correct position, It'll always keep the list sorted up to the current element. To get the median, if the number of element is odd, the median is the middle element. If the number of element is even, the median is the average of the two middle element. The total time complexity is $O(n^2)$, Idea/Brute Force: We are going to use a queue so that the first numbers we have read/enqueued are the ones we using insertion sort. e use to calculate the median - Assume that there is a queue that stores decimal numbers (I don't want to assume integers), it holds the number we have seen so far in the stream.

		- Assume an integer variable "numOfNums" exists, which holds the number of numbers we have added to the queue, aka the length of the queue. - Assume this function's arguments are both the queue of numbers and the numOfNums variable. 1. First, check if the queue is empty: a. If so, return 0 2. Check if the numOfNums variable is 0: a. If so, return 0 3. Make a decimal variable to store the sum of the numbers held in the queue. Initialize it to 0. 4. In a while loop, iterate through the queue until you reach NULL: a. Add the number you are currently on inside the queue to the sum. 5. Return the result of the sum divided by the numOfNums variable. Running Time: 1,2,3=$O(1)$, 4=$O(n)$ where n is the length of the queue, 4a=$O(1)$, 5=$O(1)$; Total Running time = $O(n)$ // idea Use two heaps(maxHeap for lower half of numbers, minHeap for upper half). To add an int from the stream, if int is less than or equal to the largest number in maxHeap, add it to maxHeap. If int is greater than the largest number in maxHeap, then add the number to minHeap. To ensure heaps are balanced : If maxHeap contains more than one extra element compared to minHeap, move the largest element from maxHeap to minHeap. If minHeap contains more elements than maxHeap, move smallest element from minHeap to maxHeap To find the median : If the sizes of maxHeap and minHeap are equal, the median is the average of the largest number in maxHeap and smallest number in minHeap. If maxHeap contains more elements than minHeap, the median is the largest number in maxHeap // runtime : Finding Median is $O(1)$, adding number from stream to heap $O(\log N)$, rebalancing heaps is $O(\log N) + O(\log N)$ because moving element from one heap to another.
12	12. A majority element in an array is an element that appears more than $n/2$ times (where n is the size of the array). Give an algorithm to find a majority element. More generally, give an algorithm to find all elements	Idea: Create a hashmap and iterate through the array. At every index add the value of the index as a key to the hashmap with a value of 1 or if the key is already in the hashtable add one to the value then retrieve all keys with value over $n/2$ This would have $O(n)$ runtime because you look through every element in the array $O(n)$ and getting something from a hashmap is an $O(1)$ runtime. Another idea for $> n/2$ times (Brute Force):

	that appear more than $n/3$ times. Use $O(1)$ extra storage.	- Assuming that the majority element exists - Sort the array with Quicksort - The majority element is now $\text{floor}(n/2)$ If sorted w/ Quicksort, the performance would be $O(n \log n)$. Note that quicksort is in-place, and thus assuming that function space does not count, $O(1)$ extra storage Idea of $> n/3$: There can only be at most 2 majority elements of $n/3$. Thus: - Initialize an array of max size of 2 - Let threshold = $\text{floor}(n/3)$ - Sort the array - Thus, the majority element(s) can be found by seeing how many times the number(s) repeat, and if it is greater than the threshold then append it into array of found majority elements $O(n \log n)$ time and $O(1)$ storage (Quicksort is in-place and the array allocated only stores 2 elements at most)
13	13. Given an array of integers containing all the elements from 1,2,...,n except one of them. Find which one. Try doing it using $O(1)$ memory. What if two elements were missing?	****SOLUTION 1: Only works for one missing element:**** Calculate the sum of all integers from 1 to n = expected sum - Use $n(n+1)/2$ formula for $O(1)$ time Calculate the sum of all elements in the array (for loop) = actual sum - This is a for loop so $O(n)$ time Subtract actual sum from expected = missing element - $O(1)$ time Return missing element - $O(1)$ time As this is using a for loop, the entire program runs at $O(n)$ time instead of $O(1)$ ****SOLUTION 2: For multiple missing elements**** For loop through the array - $O(n)$ time Initialize a checker variable to 1 - $O(1)$ time Check if $A[i] == \text{check variable}$ - $O(1)$ time If it is equal, do nothing - $O(1)$ time If it is not equal, print/append checker variable value to answer array Increment checker variable by 1 - $O(1)$ time If you were printing, then the code is done, but if you appended to an array, return the array - $O(1)$ time

		As this is using a for loop, the entire program runs at $O(n)$ time instead of $O(1)$
	14. Given an array of integers, determine if it contains a Pythagorean triple (integers a, b, c such that $c^2 = a^2 + b^2$).	Pythagorean triples: (3,4,5), (5, 12, 13), (8,15,17), ... Very brute force solution: Check all integers in arrays thrice. Yea basically check the below idea Brute force ($O(n^3)$): We have an int array (int_arr). Create a triple nested loop where $a < \text{int_arr size} - 2$, $b < \text{int_arr size} - 1$, and $c < \text{int_arr size}$. Inside the triple nested loop, if $\text{int_arr}[a]^2 + \text{int_arr}[b]^2 = \text{int_arr}[c]^2$ or $\text{int_arr}[a]^2 + \text{int_arr}[c]^2 = \text{int_arr}[b]^2$ or $\text{int_arr}[b]^2 + \text{int_arr}[c]^2 = \text{int_arr}[a]^2$, return true, otherwise return false. another idea: found a set of equations that maybe makes it better than brute force https://en.wikipedia.org/wiki/Pythagorean_triple#Special_cases_and_related_equations iterate through the list with variable a iterate through the list with variable i b and c are -1 (so you can tell if not found) if a is odd: if $i == (a^2 - 1)/2$ then $b = i$ if $i == (a^2 + 1)/2$ then $c = i$ if a is even: if $i == (a/2)^2$ then $b = i$ if $i == (a/2)^2 + 1$ then $c = i$ if b and c are both more than -1 then return (a, b, c)
15	15. The input is a sequence of ranges $[x1, y1]$, $[x2, y2]$, ... (where each $x_i \leq y_i$). Given two ranges that intersect, merge them into a single range by taking the union of ranges. Keep doing this until you finally get disjoint ranges. Design an algorithm that determines this output.	Possible recursive solution(incomplete): Sort the sequence of ranges by min x val to max x val. Simplest problem: We have two ranges $[x1, y1]$ $[x2, y2]$ Given the range $[x1, y1]$ and $[x2, y2]$ intersect, create a new range $[x, y]$ where $\min(x1, x2)$ is the new x and $\max(y1, y2)$ is the new y. Return the single combined range. Given there is no intersection we return both ranges.
16	16. Given three arrays in non-decreasing order, find the elements that are common to all of them. Try $O(1)$ extra storage.	Brute force ($O(n)$ time complexity): Create an array for common elements (called com_el_arr). Initialize 3 integers (i, j, k) as counters for the indexes of their respective arrays. While i, j, k are not equal to the size of their respective arrays, if the current index value for each of the arrays are all equal, add that value to com_el_arr. Otherwise, since the

		arrays are in non-decreasing order, if one index value (i,j,k) is less than both of the others, increment it. Return com_el_arr.
17	17. Consider two sorted arrays that are identical, except one array has an extra element. Find that element and its index.	- Initializing variables to store array lengths and the index of the extra element - Initialize the extra element index to -1 as default - Initialize pointers to traverse both arrays - While loop that checks that the pointers of both arrays are less than the end of the shorter array - If check of the current elements are different - If they are, mark the index of the extra element and exit the loop - Update pointers to the next element in each array - If the extra element is not found yet, it's the last element of the longer array - Output the extra element and its index Save the longest and shortest array. Iterate through the shortest array and compare each element, if they're not the same return the element of the longest array. If there's no difference when it reached the end of the shortest array, return the last element and index of the longest array. O(n)
18	18. Given a sorted array, find the closest element (in terms of absolute difference) to a given number.	Brute Force (O(n)) Suppose you have an array arr. Create a variable that will store the closest val index and another to store the min difference of the closest element. Using a loop, go through the entire array and calculate the difference between the number and the current index, if it is smaller than the smallest difference, update the difference and store the index of the value in the closest val index variable. Return arr[index of min difference value].
19	19. Given an array, a local maximum is an element A[i] such that is greater than or equal to the neighboring elements in the array. (There can be two neighboring elements, or just one if i = 0 or i is the last index.) Find a local maximum.	Brute Force (O(n)) Suppose you have an array A. First we need to check if the array is not empty, if it is return NULL. If the length of the array is 1, return A[0]. Then, if A[0] > A[1], return A[0]. Otherwise, initialize a counting variable that stores the current index, the previous index (i-1), and the next index (i+1), then using a while loop that iterates through the array and checks if the current value of the array is greater than both the next and previous elements. Increment that counting variable. If found, break the loop and return A[current index] because that is the first local max.
20	20. In a 2D array, a local maximum is defined as before, but the neighbors are both	Brute Force (O(n)) Suppose you have an 2D array A. First we need to check if the array is not empty, if it is return NULL. If the size of the array is 1, return A[0][0]. Then, if

	vertices and horizontal. So the neighbors of $A[i][j]$ are $A[i-1][j]$, $A[i+1][j]$, $A[i][j-1]$, $A[i][j+1]$ (whenever they exist). Find a local maximum.	$A[0][0] > A[0][1]$ and $A[0][0] > A[1][0]$, return $A[0][0]$. Otherwise, initialize 2 counting variables that store the current horizontal (col) and vertical (row) value of the array, the previous index (col-1) and (row-1), and the next index (col+1) and (row+1). Then using a while loop that iterates through the array and checks if the current value of the array is greater than both the next and previous elements. Increment that counting variable. If found, break the loop and return $A[col][row]$ because that is the first local max.
21	21. Find if a singly linked list has a loop. Use $O(1)$ extra memory.	Brute force: Assume that there is a field that holds an integer variable "n" containing how long the singly linked list is. 1. First, check if the list exists by checking if the pointer to the list is NULL. a. If it is NULL, return False. 2. Make a variable to hold the current node and set it to NULL. 3. Loop through the linked list n times to get to the last node. a. For each loop, set the current node to the next node. 4. Once there, check if the next field of the node is null. a. If it is, then it isn't a loop. Return False. 5. Otherwise, it is a loop so return True Running time: 1=$O(1)$, 2=$O(1)$, 3=$O(n)$, 3a=$O(1)$, 4=$O(1)$, 5=$O(1)$. Total run time = $O(n)$ Another brute force: 1. Let p be a pointer to the starting node 2. Iterate through linked list a. If while iterating, node pointer is ever equal to p, we have looped back, so return true b. If while iterating the node pointer is NULL, return false Should be just $O(n)$ running time as you're just running thru the LL This wouldn't work because the loop may occur in another node Another idea: - Keep two pointers fast and slow initially pointing to starting node - The fast pointer will traverse through the list faster than the slow pointer - The idea is that if the linked list is a loop, the fast pointer will eventually somehow be equal to the slow pointer, which wouldn't ever happen under a non-looped linked list
22	22. Going beyond the problem above, find the first node on the	Brute force: Assume we are given a pointer that points to the very first node of the singly linked list.

	loop (assume there is a single loop). Use $O(1)$ extra memory.	1. Check if the pointer we are given is NULL a. If so, return NULL 2. Return the address of the pointer we are given. Running time: $1=O(1)$, $2=O(1)$. Total running time = $O(1)$ -
23	23. Consider the “stock market” problem. There is an input array A of prices, of (say) one unit of stock. You need to find out the maximum profit that can be made, assuming you buy (one unit of) stock on a day and sell it on a subsequent day.	You can calculate create another array with the price differences of each day. Now this problem is transformed into a maximum-subarray problem; that is, if you get the maximum-subarray of this price difference array, you get the maximum profit. Student response: A delta array would work here, as long as you account that the max profit can incur over multiple days, which is accounted for by finding the maximum subarray.
24	24. The stock market problem again. Except, you need to determine the best selling day, for every single day. That is, for each i, find $j > i$ that maximizes $A[j] - A[i]$.	Initialize an array the same size as array A called best sell arr . Set a max price value equal to the last element in the array A ($A[-1]$). Loop through A backwards, starting from the last element to the first. Check if the max price is greater than the price for the current day, if so, set the current best sell arr index equal to the max price, otherwise set it equal to -1, signifying there is no best sell day. Set the max price to the the current price if it's greater than the current max price. Return the best sell arr. $O(n)$ time complexity.
25	25. Every array A induces a permutation π where $\pi(i)$ is the final location of the element $A[i]$ in the sorted version of A. Thus, element $A[i]$ ends up at index $\pi(i)$ after sorting. Given A, determine this permutation π.	- Let M_1 be an ordered map from i to $A[i]$ - Sort M_1 based on its values (argsort) - let M_2 be another ordered map from i to the keys of M_1 - Sort M_2 based on its values. - return the keys of M_2 - Time: $O(n \log(n))$, space: $O(n)$ Another idea: Assuming elements in A are all unique - Let π be an array of $\text{len}(A)$ - Let B be the sorted version of A - Loop through A - Find value $A(i)$ in B, keeping track of its index - Append this index to π - Now you have the permutation π Time complexity: $O(n^2)$ + sorting time, due to nested looping

26	26. Consider a version of MergeSort that does a 1/3-2/3 re split, instead of in halves. Meaning, the “left” subarray will be the first third of elements, and the “right” array is formed by the remaining elements. What is the running time of this version of MergeSort?	Since MergeSort still recursively splits the array until there's only one element in each subarray, the time complexity of MergeSort is still $O(n \log n)$.
27	27 Given a sorted array A of distinct integers, determine if there is an index i such that $A[i] = i$.	Assuming array A is sorted in ascending order: Loop through the entire array. Let's call the iteration variable i. For every iteration, check if $A[i]$ is equal to i. If they are equal, return true. If $A[i]$ is greater than i, we return false. We return false here because the minimum value the next element can be is $A[i] + 1$, and its index will be $i + 1$, so $A[i+1]$ will also be greater than $i+1$. All items are distinct, so this holds for every element after i. Finally, if the entire loop was completed without returning, no such i was found, so we return false. Worst case: $O(n)$ - When there is no i such that $A[i] = i$, and $A[k] \leq k$ for all $0 \leq k \leq \text{length}(A)$ Otherwise $O(k)$ when there is an i.
28	28 There is an extremely large array A such that the first n entries are positive integers, and all remaining entries are 0. Find n.	Set two pointers, low and high, initially pointing to the first and last elements of the array respectively. While low != high - Calculate mid = (low + high) / 2 - If $A[\text{mid}]$ is non-zero, reassign low pointer to mid + 1 - If $A[\text{mid}]$ is zero, reassign high pointer to mid Return low This algorithm uses binary search, which cuts run time in half. As such, the program's run time can be describes as $O(\log n)$.

29	Consider an $n \times n$ 2D array where all rows and columns are in sorted order. Equivalently, the following holds for any i, j : for $i' > i$, then $A[i'][j] > A[i][j]$, and for $j' > j$, $A[i][j'] > A[i][j]$. Given an x , find if x is in the array.	Brute force ($O(n^2)$): Suppose we have a $n \times n$ 2D array called A. Set a rows variable equal to the size of the array (first row) and a cols variable equal to the size of array[0] (or just set cols = rows since it's $n \times n$). Use a double nested loop (for i in rows, for j in cols), if $A[i][j] == x$, then return true. Otherwise if it's never found, then return false.
----	---	--

Data Structures Questions

No	Question	Ideas
1	1. Describe a stack based data structure that supports push, pop, and find- min in $O(1)$ operations.	- "Stack-based", so you can keep track of the minimum value as part of the data structure - The question does not disallow duplicate value - Sorting, bin heaps consequently are disallowed due to $O(1)$ requirement (Does push and pop both need to be $O(1)$)? - Bin heaps are not allowed either way (Stack) - Only findmin() needs to be $O(1)$ (a piazza post answered this) Possible solution 1: Keep a second stack to track minVal Possible solution 2: Somehow keep track of the minVal as well as the previous minVal (for popping) with only one stack - Duplicates make this hard Possible solution 3: At push store current min val in variable and push to stack the value of $2 * \text{new min val} - \text{old min val}$. Then on pop, check if

		popped value from stack is less than min val, if so return the min val and set the new min val to $2 * \text{popped min val} - \text{popped val from stack}$, else return popped value. Then for get min just return min val Keep a stack with a sorted order from max to min, with min on top. On push, maintain this order by popping the stack from min to a temp stack until the new value is bigger than the current value popped. Push new value to first stack, and move everything from temp stack back into first stack. Peek would then reveal min value in $O(1)$.
2	2. Given the pre-order traversal of a binary search tree, reconstruct the tree.	Idea: Assume pre-order traversal is given as an array PreOrder. Start by defining first (root) value of your BST as PreOrder[0]. - In a wrapper function, you could first store a copy of the first element as an int temp. You could <u>also store (could not instantiate in function. It would keep being redefined as NULL)</u> a value that is instantiated as NULL for root_right_child. When this value is NULL, we construct the left subtree. - Keep recursively defining the left subtree by taking in PreOrder[i] until you encounter a value greater than your temp value (root). - If PreOrder[i] is greater than temp, store this value in root_right_child, then begin to recursively define the right subtree. - A helper recursive function could take in root's immediate left or right child as a parameter as well as PreOrder[i] and compare values, placing them in their respective position. Brute Force: Assume pre-order traversal is given as an array - Reimplement BST insert() function - Iterate thru entire array ($0 \dots \text{len}(A)$) and call insert(A[i]) insert() takes $O(\log(n))$ time, calling n times, thus performance is $O(n \log(n))$
3	3. Design a queue using only stacks. It must support the enqueue and de-queue operations. If you start with an empty queue, end up enqueueing and dequeuing k elements, what is the total running time using your data structure?	To make a queue using only stacks, with enqueue and dequeue elements. - Note that enqueue will work like a push element (but at the end of the stack) - The dequeue is like popping the top element - asFor this solution you need to work with two stacks, one that will act like a queue and the other holds the elements everytime you push in a new element Starting with adding in elements with the idea of an enqueue, you would make two stacks (S1 and S2). When you need to add a new element it's going to be added to the button of S1, so we need a way to have it be the "first" element to be pushed. Till S1 is empty, push every element that in that stack into S2 and pop S1. Once S1 is empty, push the new element into S1. Finally put everything back from S2 in S1, and empty all of S2 so that its ready to hold for the next added element. For the idea of dequeue, it should work like a normal pop of the top element in that stack. At this moment S1 is the mock queue, popping S1 would be

		dequeuing a queue. For the run time, everything that we push in an element its $O(n)$ time because we are pushing the element in an n amount of times; and with the pop/dequeue we are following a $O(1)$ time as we don't need to traverse the stack to pop the top element.
4	4. Given login/logout time of all users for a particular website in the form: (userId, login time, logout time). Store this data, to query the total number of users who logged in and logged out in a given time range.	Slow answer: Linked List Each node would contain a time value and whether it tracks a login or logout. A node would be added to the linked list for every login and logout at a given time. To query have a counter i for logins and a counter j for logouts. Iterate through the linked list. If time is within the query range for a given node then add to the specified counter given the node is a login or logout node. If time is greater than the query range then stop counting. $O(n)$ Faster answer: AVL tree implementation: Each node contains its children, parent, and time. Have two different AVL trees one for logins and one for logouts. We would do a separate insert (and balancing) for every instance of logins and logouts to their specified tree. Do a range query for the time range on the two trees (same as agn3). $O(n \log n)$
5	5. I am designing a media player that should store songs and play them in random order. I need to be able to insert, delete, and play a random song. What data structure would you use?	Possible answer: an array or vector has indices so a pseudo random number generator function could be used for the random function and with arrays deletions are easy and insertions are only hard if not inserting at the end insert: $O(1)$ delete: i think $O(n^2)$ random: $O(n)$ Store songs in an array($O(1)$). To delete, iterate through the songs and remove the song using the method of swapping with the last element and removing last element($O(n)$). Randomly access songs to play. ($O(1)$)
6	6. Given two pointers to nodes x and y in a binary search tree,	Start at the root of the tree: <code>//(Tree* root, Tree* x, Tree* y){</code> Traverse through the entire tree

	find the unique path between them. What is the time complexity of your algorithm?	- Check if the values of both x and y are smaller than the value of the current node being looked at - If so, then move over to the left child // $root = root \rightarrow left$ - Check if the values of both x and y are greater than the value of the current node - If so, then move over to the right child // $root = root \rightarrow right$ - If one node is on the left and another node is on the right, or if one of them matches the current, if so, then that is the LCA node we are looking for, otherwise return an error or NULL if the root is not found Now find the path from LCA to the node, x Keep track of each node visited in the tree until you reach node x Now find the path from the LCA to the node, y Keep track of each node visited until you reach node y Combine the paths: - Find the unique path from x $\rightarrow$ y that consist of the path from x to the LCA and vice versa (LCA $\rightarrow$ y) - If the LCA appears at the end of the first path and beginning of the second path, then include it only once - Idea somewhat similar to above: To simplify steps, assume you're given the root node of the tree as well. The key idea is that the unique path must always at least contain the LCA of x and y. Thus if you find the LCA, you can always reconstruct a path from x to y. - Finding LCA at worst requires traversing through the height of the tree. $O(H)$ - Reconstructing a path also at worst requires traversing through the height of the tree. $O(H)$ - Thus, the runtime for this algorithm is $O(H)$. Since the height is not specifically said to be height balanced, H can be up to n, where n is the total number of elements.
7	7. You have a stream of words. Design a data structure that at any point, tells you the 10 most repeated words.	Brute force method: Initialize array storing values of the stream of words. Then iterate over the array $O(n)$ and initialize matrix array, populating it with arrays of pairs where the first value is the index of a word and the second value is its frequency, incrementing it as you go if the current word matches the word at any of the words at the index stored in matrix $O(n^2)$. Finally, sort matrix arrays based on frequency values using merge sort $O(n \log n)$. For any new words, simply repeat the previous process with insertion sort ($O(n)$ for already sorted array/matrix) and define a function that returns the first 10 words in the matrix by accessing their indices in the primary array ($O(1)$). By creating a max heap, extracting the max 10 times would give you the 10

		most repeated words. Store each node as a word with repeat count.
8	8. You have a stream of integers. Maintain the value of the median. What about the kth smallest element?	Brute Force ($O(n \log n)$) - Median Initialize an array of integers called A. Add current int to the array ($O(n)$). Sort A (For example, merge sort, $O(n \log n)$). If the number of ints in A is odd, return the middle element. Otherwise, return the average of the two middle elements ($O(1)$). Time complexity is $O(n \log n)$. Kth smallest element Initialize an array of integers called A. Add current int to the array ($O(1)$). Sort A (For example, merge sort, $O(n \log n)$). Loop through the list and return the kth element, $A[k-1]$ ($O(1)$). Time complexity is $O(n \log n)$.
9	9. A pair of nodes x,y in a (supposed) binary search tree violate the BST property if x is an ancestor of y, and the corresponding values are "out of order". Given a BST, find the number of pairs that violate the BST property.	Brute Force ($O(n^2)$) Initialize an empty array for the visited nodes and a counter of violating pairs. Traverse the tree using DFS. As you visit each node, add the key value to the array, if the current node value is less than any value in the visited nodes array, increment the counter. Once the traversal is finished, return the counter. Another idea: Assume you're given the root node of a BST. Walk through the BST using inorder search. Retrace through the BST walk, and if there are any elements that are less than the previous element ($A[i] < A[i-1]$), then that pair is out of order. Runtime: $O(H)$ [height of tree], Space: $O(n)$ (could see potential for in-place $O(1)$ but idk how to do it lol)
10	Given a string of parentheses/brackets, check if it is valid (meaning that every opening can be matched to a closing). Thus, $((\{ \{ (\}) \})) ()$ is valid, but $() (\{)$ is not.	Traverse the string, delete valid pairs, if characters remain string is not valid To add onto the previous thought, you would iterate through the string and with each "left" paran/bracket you would add it to the stack, from there if you see a right pare/bracket then you would pop from the stack and continue until you have reached the end, if the stack is not empty then you know you don't have a valid string because a valid string would have each pair pushed and popped. A valid pair being "()" or "[]" or "{}". If you have "{()}" Then you push the "{" and "(" and then when you encounter a right facing paren/bracket you pop, and since it is a valid string in the example, you would pop twice and the stack would be empty, showing that you have a valid string
11	11. An array is k-sorted, if all elements are misplaced by at most k positions (with respect to the true sorted order). Sort a	Idea: Modified heapsort For every k elements in array of length n, create a min-heap of size k. Then call extractMin() on every single heap created until all heaps created are depleted. Runtime:

	k-sorted array in time $O(n \log k)$.	- Loop for building heaps: $O(n/k) * O(k)$ [loop skips over k elements, build-heap() takes $O(k)$ time] - To build a sorted array, extract-min() will be called n times: $O(n \log(k))$ - Thus runtime is $O(n \log(k))$
12	12. Convert a max heap to a min heap.	Since you're switching the invariant, pretty much the entire array violates the invariant now - Basically just call min-heapify() to restore the (min-heap) invariant - Note that min-heapify assumes the children are correct, so you would have to restore starting at the end of the array
13	13. Consider an input of k sorted arrays of integers, each of size n. A range [a, b] intersects an array, if it contains an element of the array. The size of this range is $ b - a $. Find the smallest range intersecting all arrays.	Incomplete, does not find the <i>smallest</i> range when $\text{largestMin} < \text{smallestMax}$. Ex: input arrays {1, 3, 4, 5}, {2, 3, 4, 6}, {3, 4, 6, 7} will return range [3, 5], but [3, 4] is also acceptable. Let's create two variables, smallestMax and largestMin. Set smallestMax to the largest (last) element of the first array, and set largestMin to the smallest (first) element of the first array. We now loop over all of the arrays. For each array, we do the following: If the largest element of the array is less than smallestMax, we set smallestMax to that element. If the smallest element of the array is greater than largestMin, we set largestMin to that element. After the loop, we check our variables. If smallestMax is less than largestMin, we return the interval [smallestMax, largestMin]. Otherwise, largestMin is less than smallestMax, so we return [largestMin, smallestMax]
14	14. Augment a BST, so that it keeps track of the height of every node. Figure out how to update these values on any insert or delete.	idea: have each node hold an int for height, this will add to the constant multiple of extra space so still $O(n)$ total space. inserts into a regular binary search tree are always leaves so when inserting set height to 1. for delete the only heights that change are any "above" the one that gets deleted. using a recursive delete function decrement a node's height by one after all recursive calls so that it will happen when the code moves back 'up' the stack
15	Consider a BST. We want an extra operation "less-than delete": given x, this operation should delete all keys at most x. Design an efficient algorithm for this problem. You are allowed to augment your tree with extra $O(1)$ information per node.	idea: in-order traversal where the operation called in between left and right recursions is if key less than x: delete element (passed as a pointer so delete doesn't have to search). $O(n)$ (worst case all values are less than x)